

Laboratorio 3 — Red Team + Blue Team:

Aplicación Web Pública por HTTP:

Construir, Atacar, Detectar, Corregir y Verificar

Prototipo: CrowdStrike Falcon Alert Simulator

Brayan Loaiza Leal, Juan Sebastián Guayazán Clavijo

Resumen—Este documento presenta la primera entrega del Laboratorio 03 de Fundamentos de Seguridad de la Información, correspondiente al reto Secure Product Challenge. El equipo seleccionó la Opción 1 (automatización de incidentes de CrowdStrike Falcon) y desarrolló un prototipo denominado *CrowdStrike Falcon Alert Simulator*: dos microservicios Flask (`alerts-api` y `actions-api`) que simulan la recepción, consulta y registro de acciones sobre alertas ficticias de seguridad, junto con un dashboard estático servido por Nginx. La primera versión establece una línea base deliberadamente insegura y controlada — HTTP sin TLS, sin autenticación — utilizando únicamente información ficticia, desplegada sobre una VM Ubuntu Server 26.04.1 LTS en VMware Workstation. El trabajo comprende la descripción del problema, la arquitectura implementada, la estructura del repositorio, las evidencias de ejecución local y del servidor, un fallo real de despliegue encontrado y corregido (bloqueo por CORS entre el dashboard y la API), y un modelo inicial de amenazas STRIDE con una hipótesis por cada una de las seis categorías, derivada directamente del código fuente de los microservicios. Las pruebas y evidencias presentadas se limitan al entorno autorizado para el laboratorio y utilizan exclusivamente datos ficticios.

Keywords—HTTP, Nginx, Flask, Red Team, Blue Team, STRIDE, Threat Modeling, CrowdStrike Falcon, microservicios, Ubuntu Server.

I. DESCRIPCIÓN DEL PROYECTO

I-A. Problema seleccionado

Opción 1: automatización de incidentes de CrowdStrike Falcon. La recepción, clasificación, enriquecimiento y escalamiento de alertas de seguridad suele requerir actividades manuales que aumentan el tiempo de respuesta, dificultan la correlación de evidencias y generan criterios distintos al priorizar incidentes. Para este laboratorio se seleccionó la construcción de un prototipo denominado *CrowdStrike Falcon Alert Simulator*, diseñado exclusivamente para fines académicos y utilizando información completamente ficticia, sin integración real con la API de CrowdStrike.

I-B. Objetivo de la primera versión

Construir y publicar, mediante HTTP y sin autenticación, un prototipo que reciba alertas ficticias de CrowdStrike Falcon, permita consultarlas mediante una API REST y registre las acciones de respuesta ejecutadas por analistas. Esta versión establece una línea base técnica y una superficie controlada para las posteriores actividades de Red Team y Blue Team.

I-C. Usuarios previstos

Analistas SOC (Blue Team) que consultan el dashboard y registran acciones de respuesta sobre las alertas; el equipo Red Team, que ejercerá reconocimiento autorizado (Nmap, curl, ZAP) sobre la superficie expuesta en la Parte II desde una estación Kali Linux.

I-D. Datos ficticios

El prototipo utiliza únicamente información creada para el laboratorio:

- Cinco alertas precargadas (ALT-001-ALT-005): malware, movimiento lateral, escalamiento de privilegios, fuerza bruta y exfiltración de datos.
- Tres acciones de analista precargadas (ACT-001-ACT-003).
- Usuarios y hosts ficticios: john.doe, jane.smith, carlos.garcia, maria.torres, contractor01; WORKSTATION-42, SERVER-DC01, entre otros.

No se utilizan credenciales, tokens, información personal, direcciones internas reales ni información perteneciente a sistemas de producción.

I-E. Alcance y exclusiones

El alcance de esta primera versión comprende:

- Dos microservicios REST en Flask (`alerts-api` en el puerto 5000, `actions-api` en el puerto 5001).
- Un dashboard estático servido por Nginx que consume ambas APIs.
- Publicación mediante Nginx sobre Ubuntu Server, protocolo HTTP sobre el puerto TCP 80, con proxy inverso hacia los microservicios.
- Registro de eventos mediante los logs de Nginx y de cada microservicio.

- Documentación de la arquitectura, los flujos de información y la estructura del repositorio.
- Elaboración de un modelo inicial de amenazas mediante STRIDE (seis categorías).

Se excluyen de esta versión: información real, autenticación y autorización, HTTPS/TLS (Laboratorio 4), persistencia en base de datos (los datos viven en memoria), integración real con la API de CrowdStrike, explotación destructiva, denegación de servicio, fuerza bruta, persistencia, y las rondas activas de Red/Blue Team con Nmap, ZAP, tcpdump y hardening formal, que corresponden a la Parte II de este mismo laboratorio.

I-F. Resultado esperado del despliegue

Al finalizar esta primera versión se dispone de un prototipo accesible desde la VM del laboratorio mediante HTTP, con Nginx y los dos microservicios correctamente configurados como servicios systemd, registros de acceso y errores disponibles para su análisis, una arquitectura documentada (incluyendo un hallazgo real de diseño corregido) y un modelo inicial de amenazas que sirve de línea base para las siguientes fases del reto.

II. ARQUITECTURA IMPLEMENTADA

La arquitectura implementada corresponde a dos microservicios Flask sin autenticación, un dashboard estático y un proxy Nginx, publicados por HTTP sobre una VM Ubuntu Server. La estación Kali Linux representará, en la Parte II, el entorno desde el cual se realizan las pruebas autorizadas de Red Team; el Blue Team observará el comportamiento del servidor mediante sus registros.

II-A. Actores

- **Analista SOC / Blue Team:** consulta el dashboard y registra acciones de respuesta.
- **Red Team / Kali (Parte II):** realiza reconocimiento y pruebas autorizadas (Nmap, curl, ZAP).
- **Cliente HTTP anónimo:** cualquier origen en la red del laboratorio; no existe distinción de privilegio porque no hay autenticación.
- **Administrador/Builder:** mantiene el repositorio, el servidor y la configuración de Nginx.

II-B. Componentes

- Nginx (proxy inverso y contenido estático, puerto 80).
- alerts-api — microservicio Flask (puerto 5000).
- actions-api — microservicio Flask (puerto 5001).
- Dashboard estático (frontend/index.html).
- Registros lab3_access.log, lab3_error.log (Nginx), alerts.log, actions.log (Flask).

II-C. Flujos de datos

Navegador → **Nginx (:80)** → **proxy_pass /api/alerts/ y /api/actions/** → **alerts-api (:5000) / actions-api (:5001)**

Hallazgo de arquitectura corregido durante esta entrega: el JavaScript del dashboard llamaba originalmente de forma directa a `http://<host>:5000`, evitando el proxy de Nginx. Al desplegar en la VM real se detectó que esa llamada cruzada de puerto era bloqueada por CORS en el navegador (Flask no envía `Access-Control-Allow-Origin`), mientras que curl sí funcionaba, ocultando el problema en pruebas de servidor a servidor. Se corrigió frontend/index.html para consumir la API vía el proxy (`/api/alerts/alerts`), quedando *same-origin*; el retest con captura de navegador confirma la corrección (Sección V).

II-D. Almacenes de datos

El prototipo no usa base de datos: los datos viven en listas de Python en memoria dentro de cada microservicio, sin persistencia entre reinicios.

```
1 ALERTS -> lista en memoria (alerts-api)
2 ACTIONS -> lista en memoria (actions-api)
```

Los registros del servidor constituyen la fuente de trazabilidad:

```
1 /var/log/nginx/lab3_access.log
2 /var/log/nginx/lab3_error.log
3 services/alerts-api/alerts.log
4 services/actions-api/actions.log
```

II-E. Protocolo y puertos

Cuadro I: Protocolos y puertos principales

Servicio	Protocolo	Puerto
Nginx (proxy + estático)	HTTP	TCP/80
alerts-api (Flask, bind 0.0.0.0)	HTTP	TCP/5000
actions-api (Flask, bind 0.0.0.0)	HTTP	TCP/5001
Administración del servidor	SSH	TCP/22

Los puertos 5000/5001 quedan expuestos directamente en la red (bind 0.0.0.0), no solo tras el proxy — un hallazgo relevante para el modelo de amenazas (Sección VII).

II-F. Límites de confianza

1. **L1:** frontera entre la red del laboratorio/Internet y Nginx.
2. **L2:** frontera entre Nginx y los microservicios Flask; debilitada porque ambos hacen bind en 0.0.0.0 y son alcanzables directamente, sin pasar por Nginx.
3. **L3:** frontera entre el navegador y la API consumida por el dashboard — corregida en esta entrega para que pase por el proxy (same-origin) en vez de saltárselo.

II-G. Punto de generación de logs

Nginx registra en `lab3_access.log` la IP de origen, fecha y hora, método, ruta, código de respuesta y User-Agent de cada solicitud; los errores del servicio quedan en `lab3_error.log`. Cada microservicio Flask registra en su propio archivo (`alerts.log`, `actions.log`) las solicitudes y respuestas mediante los manejadores `before_request` / `after_request`.

II-H. Diagrama de arquitectura y flujo de datos

La Figura 1 presenta la arquitectura implementada, identificando actores, componentes, protocolo, puertos, almacenes de datos, límites de confianza y puntos de generación de logs.

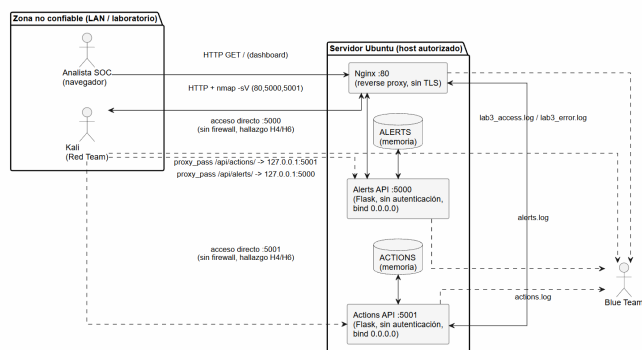


Figura 1: Diagrama de Flujo de Datos — CrowdStrike Falcon Alert Simulator.

III. ESTRUCTURA DEL REPOSITORIO

El repositorio <https://github.com/BrayanLoaiza-JuanGuayazan-FDSI/Lab3-FDSI> sigue la siguiente estructura:

```

1 Lab3-FDSI/
2 |-- frontend/index.html
3 |-- nginx/lab3-fdsi.conf
4 |-- services/
5 |   |-- alerts-api/ (app.py, requirements.txt)
6 |   |-- actions-api/ (app.py, requirements.txt)
7 |-- diagrams/dfd-lab3.png
8 |-- evidence/{local-exec.txt, server-nginx.txt, red/,
9 |             blue/}
10 |-- reports/zap-passive/
11 |-- risk-register.md
12 |-- threat-model/threat-model.xlsx
13 |-- deploy.sh
14 |-- README.md

```

El `README.md` documenta: propósito del proyecto, arquitectura, requisitos (Ubuntu 20.04+, Python 3.8+, Nginx, pip3), ejecución local, procedimiento de despliegue mediante `deploy.sh`, la URL publicada (<http://192.168.17.129/>), los integrantes del equipo y las limitaciones de seguridad conocidas (sin autenticación, sin TLS, /debug/env expuesto, sin validación de entrada, sin rate limiting, sin cabeceras de seguridad, datos en memoria, logs sin cifrar). El `risk-register.md` contiene

activos, actores, límites de confianza y superficie de ataque; la matriz de hipótesis STRIDE y de mitigaciones vive en `threat-model/threat-model.xlsx`.

IV. EVIDENCIAS DE EJECUCIÓN LOCAL

La ejecución local permite comprobar que el repositorio fue clonado correctamente, que los archivos de la aplicación existen y que el frontend funciona antes del despliegue en el servidor. El comando exacto usado en cada prueba se lista en el Anexo C.

IV-A. Estado del repositorio

```

C:\Users\jsegu\Documents\2026\FDSI\Laboratorios\Lab3-FDSI>git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

```

Figura 2: Estado del repositorio mediante `git status`.

IV-B. Historial de commits

```

jsegu@DESKTOP ~/Documents/2026/FDSI/Laboratorios/Lab3-FDSI
$ git log --oneline -5
40048db docs(report): add real http.server local evidence and appendix links
dfac8e8 docs(threat-model): move STRIDE matrix and mitigations from report to xlsx
a2fb793 docs(report): move executed commands to appendix, keep only results in body
df143a0 fix(report): widen deployment diagnostics table to span both columns
a8193b2 docs(report): add IEEE writeup for Parte I

jsegu@DESKTOP ~/Documents/2026/FDSI/Laboratorios/Lab3-FDSI
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

```

Figura 3: Últimos cinco commits del repositorio.

IV-C. Archivos del proyecto

```
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ find . -maxdepth 3 -type f
./frontend/index.html
./nginx/Lab3-fdsi.conf
./deploy.sh
./diagrams/README.md
./reports/zap-passive/.gitkeep
./README.md
./services/actions-api/requirements.txt
./services/actions-api/app.py
./services/actions-api/actions.log
./services/alerts-api/alerts.log
./services/alerts-api/requirements.txt
./services/alerts-api/app.py
./gitignore
./evidence/local-exec.txt
./evidence/red/.gitkeep
./evidence/server-nginx.txt
./evidence/blue/.gitkeep
./risk-register.md
./git/logs/HEAD
./git/FETCH_HEAD
./git/HEAD
./git/ORIG_HEAD
./git/index
./git/config
./git/description
./git/hooks/prepare-commit-msg.sample
./git/hooks/pre-rebase.sample
./git/hooks/pre-push.sample
./git/hooks/pre-merge-commit.sample
./git/hooks/pre-applypatch.sample
./git/hooks/update.sample
./git/hooks/fsmonitor-watchman.sample
./git/hooks/applypatch-msg.sample
./git/hooks/post-update.sample
./git/hooks/push-to-checkout.sample
./git/hooks/sendemail-validate.sample
./git/hooks/commit-msg.sample
./git/hooks/pre-commit.sample
./git/hooks/pre-receive.sample
```

Figura 4: Archivos presentes en el repositorio.

IV-D. Servidor HTTP local

Se sirvió `frontend/index.html` con `python3 -m http.server 8080` y se verificó con `curl -I/curl` contra `localhost:8080` (comandos completos en el Anexo C). Respuesta obtenida:

```
1 HTTP/1.0 200 OK
2 Server: SimpleHTTP/0.6 Python/3.14.0
3 Date: Mon, 14 Sep 2026 11:18:38 GMT
4 Content-type: text/html
5 Content-Length: 8294
```

El cuerpo de la respuesta contiene el `<title>Falcon Alerts</title>` del dashboard, confirmando que `index.html` existe y se sirve correctamente de forma local antes del despliegue en el servidor.

```
jsegu@DESKTOP ~/Documents/2026/FDSI/Laboratorios/Lab3-FDSI/frontend
$ python3 -m http.server 8080
Serving HTTP on 0.0.0.0 port 8080 (http://0.0.0.0:8080/) ...
127.0.0.1 - - [14/Sep/2026 06:27:25] "HEAD / HTTP/1.1" 200 -

jsegu@DESKTOP ~
$ curl -I http://localhost:8080/
HTTP/1.0 200 OK
Server: SimpleHTTP/0.6 Python/3.14.0
Date: Mon, 14 Sep 2026 11:27:25 GMT
Content-type: text/html
Content-Length: 8294
Last-Modified: Mon, 14 Sep 2026 06:17:06 GMT

jsegu@DESKTOP ~
$ curl http://localhost:8080/ | grep title
<title>Falcon Alerts</title>
```

Figura 5: Prueba de disponibilidad del frontend con `python3 -m http.server 8080` (HTTP 200).

IV-E. Fecha y hora de las pruebas

Obtenida con `date -u +%Y-%m-%dT%H:%M:%SZ`. Evidencia de ejecución local: 2026-09-14T11:18:38Z. Evidencia de despliegue en servidor: 2026-09-14T03:48:11Z.

V. EVIDENCIAS DEL SERVIDOR Y NGINX

Servidor: VM Ubuntu Server 26.04.1 LTS en VMware Workstation (red NAT), host `lab3-fdsi-ubuntu`, IP `192.168.17.129`. Los comandos exactos usados en esta sección se listan en el Anexo C.

V-A. Identificación del servidor

```
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ hostname
lab3-fdsi-ubuntu
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ whoami
fdsi
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ pwd
/home/fdsi/Lab3-FDSI
```

Figura 6: Identificación y línea base del servidor.

V-B. Versión de Nginx

Obtenida con `nginx -v`: `nginx/1.28.3 (Ubuntu)`.

V-C. Estado del servicio

```
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ systemctl status nginx --no-pager
* nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-09-14 03:48:30 UTC; 1h 3min ago
   Invocation: f6cb822bdc409c8b59f4f3726949f6
   Docs: man:nginx(8)
  Main PID: 3418 (nginx)
    Tasks: 3 (limit: 871)
   Memory: 3.0M (peak: 3.6M)
      CPU: 0ms
   CGroup: /system.slice/nginx.service
           └─3418 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─3420 "nginx: worker process"
```

Figura 7: Estado del servicio Nginx (active/running).

V-D. Validación de la configuración

Resultado de `sudo nginx -t`:

```
1 nginx: the configuration file /etc/nginx/nginx.conf
  syntax is ok
2 nginx: configuration file /etc/nginx/nginx.conf test
  is successful
```

V-E. Prueba HTTP local

Verificada con `curl -I` sobre el frontend (:80) y el health check de cada microservicio (:5000, :5001):

```
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 14 Sep 2026 04:48:17 GMT
Content-Type: text/html
Content-Length: 9050
Last-Modified: Mon, 14 Sep 2026 04:35:06 GMT
Connection: keep-alive
ETag: "6aa7797a-235a"
Accept-Ranges: bytes
```

Figura 8: Respuesta HTTP generada por Nginx (200 OK).

Código HTTP obtenido en los tres casos: 200 OK.

V-F. Configuración del servidor

El virtual host utilizado corresponde a:

```

1 server {
2     listen 80;
3     server_name _;
4
5     access_log /var/log/nginx/lab3_access.log;
6     error_log /var/log/nginx/lab3_error.log;
7
8     root /var/www/lab3;
9     index index.html;
10
11    location / {
12        try_files $uri $uri/ /index.html;
13    }
14
15    location /api/alerts/ {
16        proxy_pass http://127.0.0.1:5000/;
17        proxy_set_header Host $host;
18        proxy_set_header X-Real-IP $remote_addr;
19    }
20
21    location /api/actions/ {
22        proxy_pass http://127.0.0.1:5001/;
23        proxy_set_header Host $host;
24        proxy_set_header X-Real-IP $remote_addr;
25    }
26 }

```

V-G. Permisos del directorio publicado

Resultado de `ls -la /var/www/lab3:`

```
1 drwxr-xr-x 2 root root 4096 Sep 14 03:41 .
2 -rwxr-xr-x 1 root root 9083 Sep 14 03:44 index.html
```

Los archivos son propiedad de `root` con permisos `755/755`, legibles y ejecutables (traversal de directorio) por Nginx sin otorgar permiso de escritura al proceso del servidor `web`.

V-H. Access log

```
Extracto de sudo tail -n 50 /var/log/nginx/lab3_access.log:
```

[illegible]

Figura 9: Extracto del registro de acceso de Nginx — incluye tráfico real del navegador (192.168.17.1) consumiendo /api/alerts/alerts cada 30 segundos, correspondiente al refresco automático del dashboard.

V-I. Error log

```
Extracto de sudo tail -n 30
/var/log/nginx/lab3_error.log:
```

```
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$ sudo tail -n 30 /var/log/nginx/lab3_error.log
fdsi@lab3-fdsi-ubuntu:~/Lab3-FDSI$
```

Figura 10: Registro de errores de Nginx (vacío, sin errores registrados).

V-J. Captura del navegador

La Figura 11 muestra el dashboard ya simplificado, con las cinco alertas y los contadores por severidad correctamente cargados tras la corrección del bloqueo por CORS descrita en la Sección VII (hallazgo L3). Verificación tomada en un entorno local equivalente (`alerts-api`, `actions-api` y el proxy sirviendo el mismo `frontend/index.html` del repositorio); la evidencia contra la VM publicada (`192.168.17.129`) se actualiza en el próximo redesplicue.

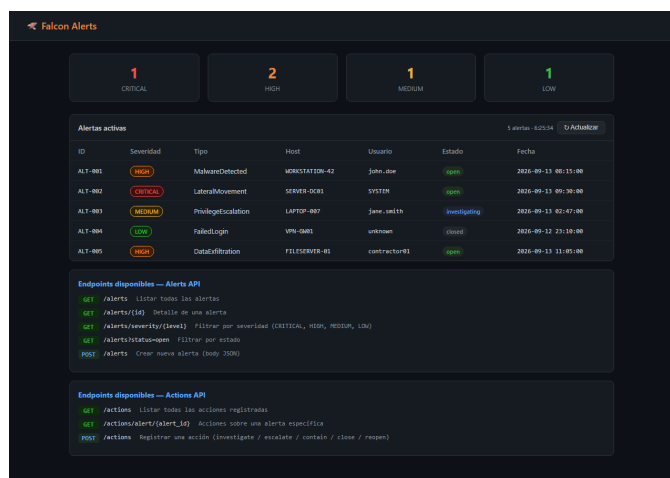


Figura 11: Dashboard de alertas cargado correctamente, 5 alertas (1 CRITICAL, 2 HIGH, 1 MEDIUM, 1 LOW).

VI. DIAGNÓSTICO DEL FALLO

El despliegue se completó exitosamente; no obstante, durante el proceso se presentaron dos fallos reales, diagnosticados y corregidos, que se documentan por su valor metodológico.

Los tres últimos casos están documentados en detalle, con evidencia de comando/log y corrección aplicada, en `risk-register.md` y en las secciones anteriores de este informe.

VII. MODELO DE AMENAZAS INICIAL

El modelo de amenazas permite identificar los activos, actores, límites de confianza y posibles riesgos de seguridad existentes en la primera versión del sistema. El detalle narrativo se encuentra en `risk-register.md`; la matriz de amenazas STRIDE y de mitigaciones, en `threat-model/threat-model.xlsx`.

Cuadro II: Diagnóstico de problemas durante el despliegue

Prueba	Esperado	Obtenido	Diagnóstico
curl localhost	HTTP 200	HTTP 200	OK, sin fallo
nginx -t	Syntax OK	Syntax OK	OK, sin fallo
pip3 install	Instalación OK	externally-managed-environment	Python 3.14/Ubuntu 26.04 (PEP 668); corregido con <code>-break-system-packages</code>
Arranque alerts-api/actions-api	active (running)	status=200/CHDIR	User=www-data sin permiso sobre \$HOME (750); corregido a usuario dueño del repo
Carga del dashboard	Alertas visibles	Error conectando a la API	CORS: fetch cruzado de puerto bloqueado; corregido consumiendo vía proxy Nginx

VII-A. Activos

- Datos de alertas (ALERTS) y de acciones (ACTIONS).
- Código fuente del repositorio.
- Configuración de Nginx y procesos alerts-api/actions-api.
- Registros (lab3_access.log, lab3_error.log, alerts.log, actions.log).
- El host (VM Ubuntu Server).

VII-B. Actores

Analista SOC (Blue Team); Red Team / Kali (Parte II); cliente HTTP anónimo (sin distinción de privilegio); administrador/Builder con acceso sudo.

VII-C. Límites de confianza

Ver Sección VII de Arquitectura: L1 (Internet/LAN → Nginx), L2 (Nginx → microservicios Flask, debilitado por bind 0.0.0.0) y L3 (navegador → API, corregido en esta entrega).

VII-D. Superficie de ataque

- Puertos TCP 80, 5000 y 5001 expuestos.
- Endpoints enumerables sin autenticación: /alerts, /alerts/{id}, /alerts/severity/{level}, /actions, /actions/alert/{id}, y el endpoint de diagnóstico /debug/env.
- Ambos microservicios corren con debug=True (Werkzeug).
- Sin server_tokens off, sin cabeceras de seguridad HTTP.

VII-E. Hipótesis STRIDE

El detalle de las seis hipótesis STRIDE (una por categoría), derivadas directamente del código de alerts-api y actions-api, junto con su elemento afectado y las mitigaciones candidatas propuestas para cada una, se documenta en la matriz de amenazas y mitigaciones threat-model/threat-model.xlsx (hojas *Threats* y *Mitigation Candidates*), incluida en el repositorio del proyecto y enlazada en el Anexo A. Cubren las seis categorías: *Spoofing*, *Tampering*, *Repudiation*, *Information Disclosure*, *Denial of Service* y *Elevation of Privilege*.

VII-F. Relación entre amenazas y arquitectura

Las seis amenazas se relacionan con la ausencia de autenticación, cifrado y validación en los dos microservicios, y con el bind en 0.0.0.0 que extiende la superficie de ataque más allá del proxy de Nginx. Los registros del servidor (access.log y los .log de cada microservicio) constituyen el principal mecanismo disponible para analizar Repudiation e Information Disclosure en la Parte II.

VII-G. Mitigaciones propuestas

Las mitigaciones candidatas (M-001 a M-006), una por cada hipótesis STRIDE, se listan junto con las amenazas que atienden en la hoja *Mitigation Candidates* de threat-model/threat-model.xlsx.

VIII. CONCLUSIONES

- Se construyó y desplegó exitosamente el prototipo CrowdStrike Falcon Alert Simulator (dos microservicios Flask + Nginx) sobre una VM Ubuntu Server 26.04.1 LTS, usando únicamente información ficticia.
- El análisis STRIDE, basado en el código real de los microservicios, identificó un hallazgo de arquitectura no evidente a simple vista: el bind en 0.0.0.0 de ambos servicios Flask y la llamada directa del frontend a la API rompían el límite de confianza L2/L3 asumido en el diseño original; este último se corrigió y se verificó con retest.
- El proceso de despliegue reveló dos fallos reales (PEP 668 en pip3, y permisos de systemd sobre \$HOME) que fueron diagnosticados y corregidos con evidencia reproducible, aportando valor más allá de un despliegue "que simplemente funcionó a la primera".
- La ejecución local y las pruebas sobre Nginx permiten establecer una línea base técnica del servicio antes de las actividades controladas de Red Team y Blue Team de la Parte II.
- Esta primera versión constituye una línea base deliberadamente insegura y controlada. Las correcciones de hardening reducen exposición pero no resuelven el riesgo de fondo de HTTP sin cifrado ni autenticación, que queda explícitamente abierto para el Laboratorio 4.

REFERENCIAS

[1] Escuela Colombiana de Ingeniería Julio Garavito, “Laboratorio 3 — Red Team + Blue Team: Aplicación web pública por HTTP: construir, atacar, detectar, corregir y verificar,” Fundamentos de Seguridad de la Información, 2026.

[2] CrowdStrike, “Alerts API Reference,” CrowdStrike Developer Portal, 2024. <https://developer.crowdstrike.com/api-reference/collections/alerts/>

[3] Pallets Projects, “Flask Documentation,” 2024. <https://flask.palletsprojects.com/>

[4] OWASP Foundation, “OWASP ZAP,” OWASP. <https://www.zaproxy.org/>

[5] OWASP Foundation, “Web Security Testing Guide,” OWASP. <https://owasp.org/www-project-web-security-testing-guide/>

[6] F5 Networks, “NGINX Documentation,” NGINX. <https://nginx.org/en/docs/>

[7] G. Lyon, “Nmap Network Scanning,” Nmap. <https://nmap.org/>

APÉNDICE A
ENLACES

- **Repositorio:** <https://github.com/BrayanLoaiza-JuanGuayazan-FDSI/Lab3-FDSI>
- **Matriz de amenazas y mitigaciones (Excel):** https://pruebacorreoescuolaingeducu-my.sharepoint.com/:x/g/personal/juan_guayazan-c_mail_escuelaing_edu_co/IQCxLx8MX5sySiz1uJl1FwUkAQMTC_xtYA3lVGlnFN1Ojvc?e=U0kC20 (también versionado en el repositorio, en `threat-model/threat-model.xlsx`)

APÉNDICE B
LISTA DE EVIDENCIAS

1	evidence/	
2	-- local-exec.txt	(git status, git log, find)
3	'-- server-nginx.txt	(hostnamectl, nginx, curl, logs)
4		
5	images/ (para Overleaf)	
6	-- dfd-lab3.png	
7	-- git-status.png	
8	-- git-log.png	
9	-- find-files.png	
10	-- curl-local.png	
11	-- server-baseline.png	
12	-- nginx-status.png	
13	-- curl-nginx.png	
14	-- access-log.png	
15	-- error-log.png	
16	'-- browser-dashboard.png	

APÉNDICE C
COMANDOS DE VERIFICACIÓN

```
1 # Repositorio (local)
2 git status
3 git log --oneline -5
4 find . -maxdepth 3 -type f
5 date -u +%Y-%m-%dT%H:%M:%SZ
6
7 # Servidor HTTP local (frontend, antes del despliegue)
8 cd frontend
```

```
9 python3 -m http.server 8080
10 curl -I http://localhost:8080
11 curl http://localhost:8080
12
13 # Servidor
14 hostname
15 whoami
16 pwd
17 nginx -v
18 systemctl status nginx --no-pager
19 sudo nginx -t
20 curl -I http://localhost
21 curl -I http://localhost:5000/health
22 curl -I http://localhost:5001/health
23
24 # Permisos
25 ls -la /var/www/lab3
26
27 # Logs
28 sudo tail -n 50 /var/log/nginx/lab3_access.log
29 sudo tail -n 30 /var/log/nginx/lab3_error.log
```

APÉNDICE D
OBSERVACIÓN SOBRE LAS EVIDENCIAS

Las capturas y resultados incluidos en este documento corresponden exclusivamente a pruebas realizadas durante el laboratorio (VM lab3-fdsi-ubuntu, IP 192.168.17.129, 2026-09-14). No se inventaron direcciones IP, códigos HTTP, timestamps, resultados de comandos, logs ni configuraciones. La IP utilizada corresponde a una red NAT interna de laboratorio (VMware), no a una IP pública de Internet.